Library Book Management System

Product Requirements Document (PRD)

Emmanuel Garcia

1. Overview

Summary: The Library Book Management System is a Python-based application designed to help users organize, search, and manage a collection of books. The project will use data structures and algorithms learned in class to make searching for books and managing reservations more efficient.

Background: Libraries need an organized way to keep track of books, their availability, and reservations. Managing this information manually can become time-consuming as the number of books increases. I chose this project because it provides a practical way to apply concepts from my Data Structures course, including lists, hash tables, queues, and Big-O analysis. The application will also include a basic visual interface to make it easier to use.

2. Problem Statement

Managing a collection of books can become difficult without an organized system. Users need a way to quickly search for books, determine whether they are available, and keep track of reservations.

Without an organized system, finding a particular book may require searching through an entire list. Managing reservations can also become confusing when multiple people are waiting for the same book.

The application will address these problems by providing a simple interface that organizes books and uses efficient data structures to manage searches and reservations.

Evidence. This project is based on common library management tasks rather than formal user research. During development, I plan to compare different searching methods and test how efficiently the application handles book records. These tests will help demonstrate the advantages of using different data structures.

3. Goals and Success Metrics

Goal	Metric
Allow users to add and organize books	Successfully add and display at least 50 sample books without errors.
Provide efficient book searching	Successfully find books using ISBN and compare list searching with dictionary searching.
Manage book reservations	Test a queue with at least five users and confirm reservations are processed in the correct order.

Create a functional visual interface	Have at least two users test main features and complete tasks without assistance.
Demonstrate Big-O analysis	Explain the time complexity of at least three operations and compare search performance using sample data.

Non-Goals. This project will not include online book purchases, payment processing, or connections to real library databases. It will also not require users to create online accounts. The main focus will be demonstrating data structures through a working application.

4. Users and Use Cases

Target Users / Personas

Library Staff: Someone responsible for organizing books and tracking their availability. They need a simple way to add books, search for the collection, and manage reservations without spending too much time navigating the application.

Library Visitors: Someone looking for a particular book who wants to know whether it is available. They need a quick way to search for books and join a waiting list if a book is currently unavailable.

User Stories

As a library staff member, I want to add new books to the system so that the collection stays organized.

As a library visitor, I want to search for a book with its ISBN so that I can quickly find the correct book.

As a library visitor, I want to see whether a book is available so that I can borrow it.

As a library visitor, I want to join a waiting list when a book is unavailable so that I can reserve my place online.

As a library staff member, I want reservations processed in the order they were received so that the system remains fair.

5. Requirements

ID	Requirement	Priority	Notes
FR-1	The system shall allow users to add books with a title, author, ISBN, and availability status.	P0	Books stored in a Python list and indexed by ISBN in a dictionary.
FR-2	The system shall allow users to search for books using their ISBN.	P0	Uses a hash table through a Python dictionary.
FR-3	The system shall display all books and their availability.	P0	Uses a list to organize book records.

FR-4	The system shall allow users to join a reservation waiting list for unavailable books.	P0	Use a FIFO queue for each book.
FR-5	The system shall allow staff to mark books as borrowed or returned.	P0	Updates book availability. When a book is returned, the system checks its FIFO reservation queue and assigns the next reservation if applicable.
FR-6	The system shall provide a graphical interface.	P1	Uses Python Tkinter.
FR-7	The system shall compare the execution time of list searching and dictionary searching.	P1	Demonstrates Big-O analysis.
FR-8	The system shall allow users to sort books alphabetically by title.	P2	Optional feature for later development.

Priority guide: P0 = must have for launch; P1 = important; P2 = nice to have.

6. User Experience

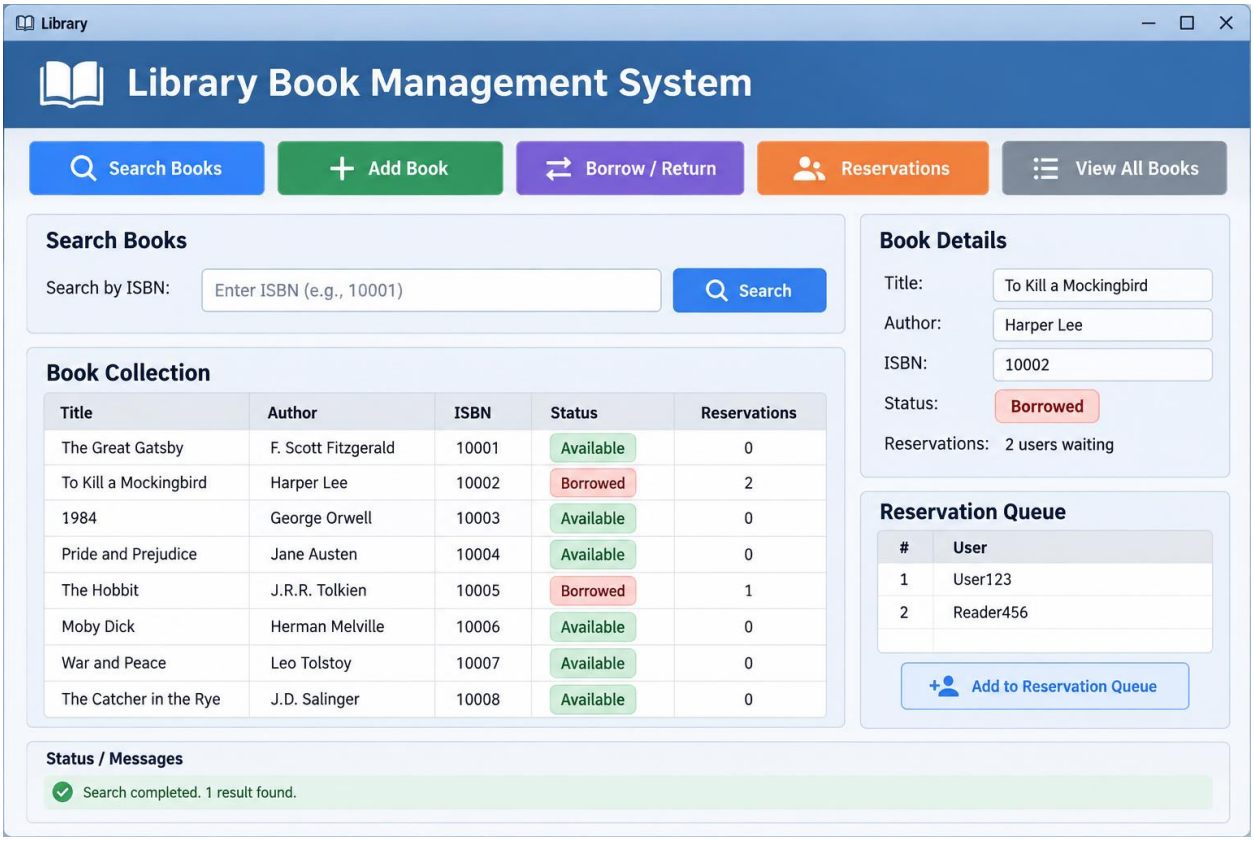
User Flow. When the application opens, users will see the main library screen displaying the available books. From there, users can search for a book, add a new book, or manage reservations.

Open Application → Main Library Menu → Add Book / Search Books / Reservations → Display Results or Confirmation

Designs. I plan to use Tkinter to create a simple desktop interface with buttons, text fields, and a table displaying book information. The design will prioritize functionality and readability over advanced graphics.

Conceptual Interface Sketch (sample data)

Edge Cases and Error States. The application will display an error message if a user searches for a book that does not exist. It will also prevent users from adding books with duplicate ISBNs or leaving required fields empty. If a reservation queue is empty, the system will indicate that no users are waiting. When a borrowed book is returned, the system will check its waiting list before making it available again.



(AI-Generated) The image above is an AI-generated mockup representing my proposed design for the Library Book Management System. It illustrates how I envision the application displaying book records, search functions, availability information, and reservation management. The final interface will be developed using Python Tkinter and may differ from this initial concept.

7. Technical Considerations

Architecture Notes. The application will be developed using Python. It will use several data structures to manage information:

Lists: Store and display the collection of books.

Hash Tables: Python dictionaries will allow quick searches using ISBN numbers.

Queues: Python's collections.deque will manage book reservations using the first-in, first-out principle.

Big-O Analysis: Compare the performance of different search operations to understand how their efficiency changes as the collection grows.

A linear search through a list has a time complexity of O(n), while dictionary lookup has an average-case time complexity of O(1). This comparison will help demonstrate the importance of choosing appropriate data structures.

Integrations and Dependencies. I plan to use Python's Tkinter library to build a visual interface. Tkinter is available in most standard Python installations. I will also use Python's built-in timeit module to measure execution times.

Data and Analytics. Each book will have a title, author, ISBN, and availability status. Reservation queues will contain sample user identifiers. For testing, I plan to use collections of 10, 100, and 1,000 books to compare search performance.

Migration or Rollout Concerns. The first version will run locally on a computer and use sample data stored in memory. This means records will be reset when the program closes. If time allows, I may add the ability to save and load book information using a JSON file.

8. Risks and Mitigations

Risk	Likelihood	Impact	Mitigation
Difficulty implementing graphical interface	Medium	Medium	Begin with a simple Tkinter window and add features gradually.
Errors managing reservation queues	Medium	High	Test reservations with multiple sample users to confirm the correct order.
Duplicate or incorrect book information	Medium	Medium	Validate input and prevent duplicate ISBNs.
Difficulty demonstrating performance differences	Medium	Medium	Use different dataset sizes and repeat timing tests.
Not completing all planned features	Medium	High	Complete P0 requirements first and leave optional features for later.

9. Open Questions

Should the final application save book information after it closes, or is temporary storage acceptable for the

assignment?

(Owner: Emmanuel Garcia; Due: October 23, 2026)

Will Tkinter provide all the interface features needed, or should I consider another library?

(Owner: Emmanuel Garcia; Due: October 23, 2026)

Should the project include an additional algorithm, such as sorting, beyond the required three topics?

(Owner: Emmanuel Garcia; Due: October 30, 2026)

10. Revision History

Version	Date	Author	Changes
0.1	October 20, 2026	Emmanuel Garcia	First PRD draft outlining the project, features, & data structures.